Full Name: Peter Bækgaard
Study Number: 20212624
Field of Study: Software - SW7

Link based ranking and impact on index

Web Intelligence - Software 7

1 Describe how and where ranking comes into the picture in basic search engine architecture

When making a search engine, we want the search results to be as relevant as possible to the query and contain pages that the user expects find. This is the purpose of the ranking algorithms. Ranking algorithms are used to rank the pages of a search to give relevant and satisfactory results that align with the search query of the user.

To rank the results, the search engine first needs a database of some webpages. These webpages are gathered by the crawler.

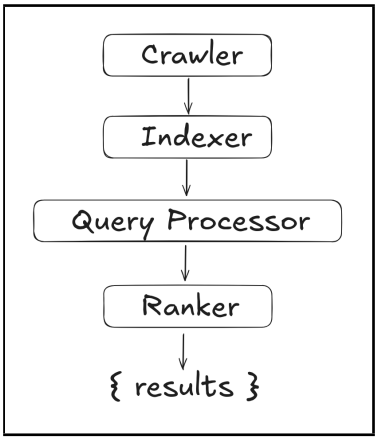

Figure 1: The high-level structure of a search engine.

The pages are accompanied by some metadata and keywords link to the pages. Where we place the ranker is highly dependant on the kind of Ranker we use. Ranking can take place in different parts of the pipeline of a search engine. figure 1 shows an example of a pipeline of a search engine. Some search engines like the one in figure 1 implements the ranker during the querying process, this is also known as "online" rankers, and are typically content-based ranking algorithms. The ranking then takes place after the user has provided a query. When a user makes a query, e.g. "How to bake potatoes", the ranker will take the query, and perform some calculation depending on the algorithm used to provide scores for each page that is then used to give results that is relevant to baking potatoes. Other There are other rankers that work by ranking after the indexer has done its job, these algorithms we call "offline" rankers and are typically link-based ranking algorithms. This is for example the PageRank algorithm. Which calculates a score for each page, which can then be stored in the index for future use for queries.

Full Name: Peter Bækgaard
Study Number: 20212624
Field of Study: Software - SW7

2 Explain and discuss your implementation of PageRank or the HITS algorithms and where you cut corners. What are the basic differences between the two algorithms?

In terms of implementing a ranker for a search engine, there are 2 classes of rankers to look at. The Content-based Rankers, and the Link-based rankers. The content based rankers, tries to rank the pages from a query, purely based on the content of the query and the documents in the database. The Link-based rankers will delimit the database from the query and rank based on a score calculated based on the authority of the pages (Quality).

For the implementation of a link-based ranker, the PageRank was the one that i have implemented.

Algorithm 1 PageRank(Index : Index after crawling, α : 0.85)

Initialize $q \leftarrow \frac{1}{N}$ for all N pages (uniform distribution)
Initialize $U \leftarrow \frac{1}{N}$ for all transitions
Initialize $T \leftarrow$ Set the transition to $\frac{1}{L}$ for all links to pages on the page, where L = number of links
Initialize $P \leftarrow (1-\alpha) \cdot T + \alpha \cdot U$
while q not converged **do**
  $q \leftarrow q \cdot P$
end while
for *Page* in *Pages* **do**
  $Page.Score \leftarrow q_{Page}$
end for

In algorithm 1 i have written a pseudo code showing the implementation of PageRank. The algorithm works by taking in the index after the crawler has finished. The crawler will find all the links on each of the pages and keep track of these. Inside the PageRank algorithm, i then initialize the uniform distribution q which contains the final scores for the pages when the algorithm is done. We create U which is the same format as the probability matrix P but with all values set to $\frac{1}{N}$. P is then set using the formula $P \leftarrow (1-\alpha) \cdot T + \alpha \cdot U$. We repeatably multiply each iteration of q on to P until convergence where we then assign the scores for each page. By using this algorithm, the ranking is performed in an "offline" manner, meaning that the ranking is before before the query. That way the results for the enduser is much faster than "online" algorithms. When the user performs a query, we use query processing to get the pages that comes close to the query and sort them by their PageRank score. Another implementation of link-based ranking that could have been implemented was the HITS algorithm. The way HITS differs from PageRank, is that HITS works in an "online" basis (after user has given a query). It takes a subset of the results from something like a content based ranker and then take all the neighboring pages to thosepages. You then keep track of two scores: The Authoritive Score, and the Hub Score. Authorive Score tells us how relevant the page is to the query and the Hub Score doesnt necessarily tell us the revelance but tells us how good it is a pointing to relevant pages. These two scores are then used until convergence where we rank on the authorative scores of the pages.

Full Name: Peter Bækgaard
Study Number: 20212624
Field of Study: Software - SW7

3 Discuss how you have stored your page rank in index and/or how it effects the index.

If we look at the different ranking algorithms, and how they affect the index, we will see a few differences in terms of implementation needs. We can see a significant difference in size of the index. A search engine that implements a content-based ranker like Vector-Space Model (VSM), would need to store the term weight and normalized term weight for each word in a page. This leads to a huge index file as each page contains a large number of words. Contrarily if we look at an algorithm like PageRank, we only need to store the outlinks of each page and the score. Further more we also need to store an index of all keywords and which pages they appear in for the query processor to delimit the pages to only show ones that contain the query words.

As for storing the ranks from the ranking algorithm previous explained (see algorithm 1), i have chosen to store the scores in the index by each of the page using a key called Score. Doing it this way, makes the implementation very easy for ranking pages for a query, as we only need to get the subset of pages that satisfy the query and rank based on the key of the pages.

If we look at the different ranking algorithms, and the way they evolve the index. We can see a significant difference in size of the index. A search engine that implements the PageRank algorithm, would only need to have an index of the keywords and their associated documents, and the documents would only need their scores. If we look at other algorithms like the VSM

4 Show examples of the results of queries also together with content ranking on the data from your crawler.

```
{
    "https://dr.dk": {
        "url": "https://dr.dk",
        "title": "DR: Nyheder - Breaking - TV - Radio",
        "Score": 0.15,
        "outgoing_links": [
            "https://dr.dk/drtv/",
            "https://dr.dk/lyd/",
            ...
        ]
    },
    "https://dr.dk/lyd/": {
        "url": "https://dr.dk/lyd/",
        "title": "Radio & Podcast fra DR - Hør netradio og podcast her | DR LYD",
        "Score": 0.09,
        "outgoing_links": [
            "https://dr.dk/",
            "https://dr.dk/drtv/",
            ...
        ]
    }
```


Figure 2: Example of Index after PageRank

To give a demonstration, a user could potential search using the following query: "DR"

The result from before ranking the result, would then be a list of sites that contain words from the query (see figure 2). The result would then simply be sorted on the "Score" key. The final top 3 result could then be something like:

1. "https://dr.dk"
2. "https://dr.dk/nyheder"
3. "https://dr.dk/drtv"

5 Compare the results of content-based ranker and link-based ranker and discuss them.

Different rankers can give different results due to their implementation and how they work. For Example, as explained in section 2 there are 2 classes of rankers, the content-based and link-based rankers. Content-based rankers like VSM. VSM works by calculating the inverse document frequency and term frequency comparing the query to documents using this. While this approach is very fast and has low overhead, it generally achieves subpar results compared to a link-based approach. The reason for this is because the results of the content-based rankers are generally relevant pages. But they do not take quality into account, which is what the link-based rankers tries to implement. Looking at the results in figure 3, we see the results from the Link-Based Ranker is better in terms of quality, while also keeping the results relatively relevant, with results a user would expect to find when searching for Princess Diana, like *royal.gov.uk* etc. On the other hand, the Content-Based Ranker also gave relevant results, we see that the results are of low quality, with results like auctions or x-rated content.

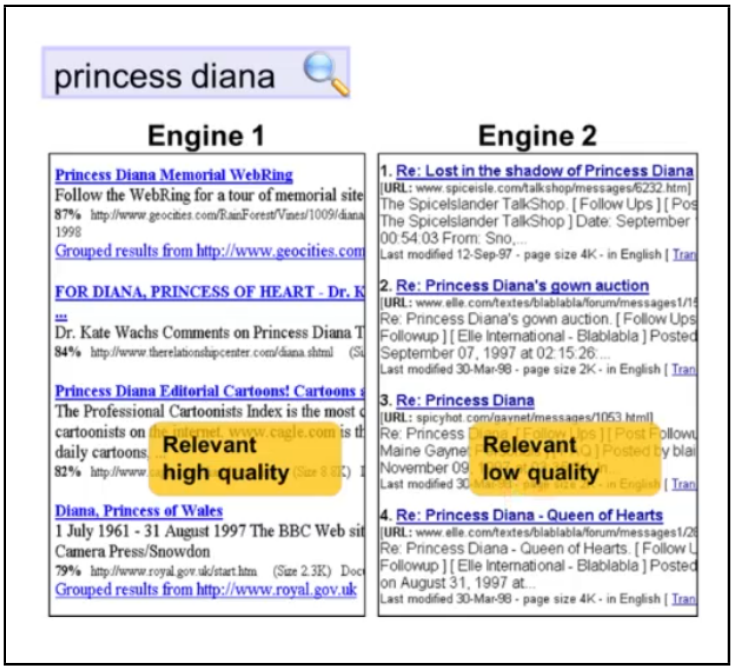

Figure 3: Example of results from different rankers [1]
Engine 1: Link-Based Ranker
Engine 2: Content-Based Ranker

6 Compare also the aggregated rank from both rankers in comparison to just single rankers

Another approach to ranking is aggregating rankers. One approach to this, is using formula (1). Using the formula, you are able to get a combined score based on two different rankers. The β value, is used to determine which of the rankers to priotitize.

Due to time constraints, i didn't have time to succesfully implement an aggregated ranker. However, i theorize that an aggregated ranker using a link-based ranker like PageRank and a content-

based ranker like VSM would yield better results compared to just using PageRank. I expect this, because this aggregated ranker would allow for fine-tuning both the quality (via PageRank) and the relevance (via VSM) of the results returned by the search engine. By utilizing an aggregated ranker combining PageRank and VSM, we are able to still prioritize high quality pages, while pushing up the most relevant of the top quality pages. I presume the results would therefore be of higher quality, as for comparing it to just using PageRank. In contrast to the aggregated ranker, the PageRank algorithm alone will only prioritize the pages that is most likely visited, delimited by the query. While the Aggregated ranker will push up the more relevant pages of these pages.

$$\text{Score} = (\beta \cdot \text{ScoreA}) + ((1 - \beta) \cdot \text{ScoreB}) \tag{1}$$

Formula 1: Formula for calculating the score by aggregating two ranking algorithms.
β: Weight to determine which ranking algorithm to prioritize, where $\beta \in [0, 1]$.
ScoreA: Score from Ranker A.
ScoreB: Score from Ranker B.

References

[1] P. Dolog. Lecture 5, Slide 1 (Introduction), Page 9.